

Module `java.base`**Package** `java.util.function`

```
package java.util.function
```

Functional interfaces provide target types for lambda expressions and method references. Each functional interface has a single abstract method, called the *functional method* for that functional interface, to which the lambda expression's parameter and return types are matched or adapted. Functional interfaces can provide a target type in multiple contexts, such as assignment context, method invocation, or cast context:

```
// Assignment context
Predicate<String> p = String::isEmpty;

// Method invocation context
stream.filter(e -> e.getSize() > 10)...

// Cast context
stream.map((ToIntFunction) e -> e.getSize())...
```

The interfaces in this package are general purpose functional interfaces used by the JDK, and are available to be used by user code as well. While they do not identify a complete set of function shapes to which lambda expressions might be adapted, they provide enough to cover common requirements. Other functional interfaces provided for specific purposes, such as `FileFilter`, are defined in the packages where they are used.

The interfaces in this package are annotated with `FunctionalInterface`. This annotation is not a requirement for the compiler to recognize an interface as a functional interface, but merely an aid to capture design intent and enlist the help of the compiler in identifying accidental violations of design intent.

Functional interfaces often represent abstract concepts like functions, actions, or predicates. In documenting functional interfaces, or referring to variables typed as functional interfaces, it is common to refer directly to those abstract concepts, for example using "this function" instead of "the function represented by this object". When an API method is said to accept or return a functional interface in this manner, such as "applies the provided function to...", this is understood to mean a *non-null* reference to an object implementing the appropriate functional interface, unless potential nullity is explicitly specified.

The functional interfaces in this package follow an extensible naming convention, as follows:

- There are several basic function shapes, including `Function` (unary function from T to R), `Consumer` (unary function from T to void), `Predicate` (unary function from T to boolean), and `Supplier` (nullary function to R).
- Function shapes have a natural arity based on how they are most commonly used. The basic shapes can be modified by an arity prefix to indicate a different arity, such as `BiFunction` (binary function from T and U to R).
- There are additional derived function shapes which extend the basic function shapes, including `UnaryOperator` (extends `Function`) and `BinaryOperator` (extends `BiFunction`).
- Type parameters of functional interfaces can be specialized to primitives with additional type prefixes. To specialize the return type for a type that has both generic return type and

generic arguments, we prefix ToXxx, as in `ToIntFunction`. Otherwise, type arguments are specialized left-to-right, as in `DoubleConsumer` or `ObjIntConsumer`. (The type prefix `Obj` is used to indicate that we don't want to specialize this parameter, but want to move on to the next parameter, as in `ObjIntConsumer`.) These schemes can be combined, as in `IntToDoubleFunction`.

- If there are specialization prefixes for all arguments, the arity prefix may be left out (as in `ObjIntConsumer`).

Since:

1.8

See Also:

`FunctionalInterface`

Related Packages

Package	Description
<code>java.util</code>	Contains the collections framework, some internationalization support classes, a service loader, properties, random number generation, string parsing and scanning classes, base64 encoding and decoding, a bit array, and several miscellaneous utility classes.

Interfaces

Class	Description
<code>BiConsumer</code> <code><T,U></code>	Represents an operation that accepts two input arguments and returns no result.
<code>BiFunction</code> <code><T,U,R></code>	Represents a function that accepts two arguments and produces a result.
<code>BinaryOperator</code> <code><T></code>	Represents an operation upon two operands of the same type, producing a result of the same type as the operands.
<code>BiPredicate</code> <code><T,U></code>	Represents a predicate (boolean-valued function) of two arguments.
<code>BooleanSupplier</code>	Represents a supplier of boolean-valued results.
<code>Consumer</code> <code><T></code>	Represents an operation that accepts a single input argument and returns no result.
<code>DoubleBinaryOperator</code>	Represents an operation upon two double-valued operands and producing a double-valued result.
<code>DoubleConsumer</code>	Represents an operation that accepts a single double-valued argument and returns no result.
<code>DoubleFunction</code> <code><R></code>	Represents a function that accepts a double-valued argument and produces a result.

DoublePredicate	Represents a predicate (boolean-valued function) of one double-valued argument.
DoubleSupplier	Represents a supplier of double-valued results.
DoubleToIntFunction	Represents a function that accepts a double-valued argument and produces an int-valued result.
DoubleToLongFunction	Represents a function that accepts a double-valued argument and produces a long-valued result.
DoubleUnaryOperator	Represents an operation on a single double-valued operand that produces a double-valued result.
Function<T,R>	Represents a function that accepts one argument and produces a result.
IntBinaryOperator	Represents an operation upon two int-valued operands and producing an int-valued result.
IntConsumer	Represents an operation that accepts a single int-valued argument and returns no result.
IntFunction<R>	Represents a function that accepts an int-valued argument and produces a result.
IntPredicate	Represents a predicate (boolean-valued function) of one int-valued argument.
IntSupplier	Represents a supplier of int-valued results.
IntToDoubleFunction	Represents a function that accepts an int-valued argument and produces a double-valued result.
IntToLongFunction	Represents a function that accepts an int-valued argument and produces a long-valued result.
IntUnaryOperator	Represents an operation on a single int-valued operand that produces an int-valued result.
LongBinaryOperator	Represents an operation upon two long-valued operands and producing a long-valued result.
LongConsumer	Represents an operation that accepts a single long-valued argument and returns no result.
LongFunction<R>	Represents a function that accepts a long-valued argument and produces a result.
LongPredicate	Represents a predicate (boolean-valued function) of one long-valued argument.

LongSupplier	Represents a supplier of long-valued results.
LongToDoubleFunction	Represents a function that accepts a long-valued argument and produces a double-valued result.
LongToIntFunction	Represents a function that accepts a long-valued argument and produces an int-valued result.
LongUnaryOperator	Represents an operation on a single long-valued operand that produces a long-valued result.
ObjDoubleConsumer<T>	Represents an operation that accepts an object-valued and a double-valued argument, and returns no result.
ObjIntConsumer<T>	Represents an operation that accepts an object-valued and a int-valued argument, and returns no result.
ObjLongConsumer<T>	Represents an operation that accepts an object-valued and a long-valued argument, and returns no result.
Predicate<T>	Represents a predicate (boolean-valued function) of one argument.
Supplier<T>	Represents a supplier of results.
ToDoubleBiFunction<T,U>	Represents a function that accepts two arguments and produces a double-valued result.
ToDoubleFunction<T>	Represents a function that produces a double-valued result.
ToIntBiFunction<T,U>	Represents a function that accepts two arguments and produces an int-valued result.
ToIntFunction<T>	Represents a function that produces an int-valued result.
ToLongBiFunction<T,U>	Represents a function that accepts two arguments and produces a long-valued result.
ToLongFunction<T>	Represents a function that produces a long-valued result.
UnaryOperator<T>	Represents an operation on a single operand that produces a result of the same type as its operand.

[Report a bug or suggest an enhancement](#)

For further API reference and developer documentation see the [Java SE Documentation](#), which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. [Other versions.](#)

Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.

Copyright © 1993, 2025, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.

All rights reserved. Use is subject to [license terms](#) and the [documentation redistribution policy](#). [Modify Cookie Preferences.](#)
[Modify Ad Choices.](#)